



Gymnázium J. K. Tyla

# **Autonomní pipeline pro vývoj softwaru**

Návrh a realizace systému DarkFactory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.c

Vedoucí práce: Jméno vedoucího práce

## **Prohlášení**

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne .....

Podpis autora práce:

.....

## **Anotace**

Práce se zabývá návrhem a realizací autonomního systému pro vývoj softwaru, který přebírá rutinní kroky vývojového procesu — od přijetí požadavku přes jeho interpretaci a naplánování až po vytvoření a ověření změny. Teoretická část shrnuje principy kontinuální integrace, řízení verzí a orchestrace jazykových modelů. V praktické části je popsán systém DarkFactory, jeho architektura a governance, a jsou vyhodnoceny výsledky jeho nasazení na reálných repozitářích.

## **Klíčová slova**

autonomní systémy, softwarové inženýrství, kontinuální integrace, orchestrace agentů, jazykové modely

## **Annotation**

This thesis deals with the design and implementation of an autonomous software engineering system that takes over the routine steps of the development process — from intake of a request through its interpretation and planning to the creation and verification of a change. The theoretical part summarises the principles of continuous integration, version control and the orchestration of language models. The practical part describes the DarkFactory system, its architecture and governance, and evaluates the results of its deployment on real repositories.

## **Keywords**

autonomous systems, software engineering, continuous integration, agent orchestration, language models

# Obsah

|                                                |    |
|------------------------------------------------|----|
| Prohlášení .....                               | 2  |
| Anotace .....                                  | 3  |
| Klíčová slova .....                            | 3  |
| Annotation .....                               | 3  |
| Keywords .....                                 | 3  |
| 1 Úvod .....                                   | 6  |
| 1.1 Cíl práce .....                            | 6  |
| 1.2 Struktura práce .....                      | 6  |
| 2 Teoretická část .....                        | 7  |
| 2.1 Řízení verzí a model pull requestu .....   | 7  |
| 2.2 Kontinuální integrace .....                | 7  |
| 2.3 Jazykové modely a jejich orchestrace ..... | 7  |
| 2.3.1 Problém jednoho poskytovatele .....      | 7  |
| 2.4 Governance automatizovaných změn .....     | 7  |
| 3 Praktická část .....                         | 8  |
| 3.1 Přehled architektury .....                 | 8  |
| 3.2 Životní cyklus požadavku .....             | 8  |
| 3.3 Popis prostředí repozitáře .....           | 8  |
| 3.4 Orchestrace napříč poskytovateli .....     | 9  |
| 3.5 Ověřování změn .....                       | 9  |
| 4 Výsledky a diskuse .....                     | 10 |
| 4.1 Nasazení .....                             | 10 |
| 4.2 Chyby, které se projeví až v provozu ..... | 10 |
| 4.3 Diskuse .....                              | 10 |
| 4.4 Omezení .....                              | 10 |
| 5 Závěr .....                                  | 11 |
| Seznam příloh .....                            | 12 |
| A Obsah přiloženého média .....                | 13 |
| B Struktura konfiguračního souboru .....       | 14 |
| Seznam zdrojů .....                            | 15 |

## **Seznam obrázků a tabulek**

|           |                                                                 |    |
|-----------|-----------------------------------------------------------------|----|
| Obrázek 1 | Logo Gymnázia J. K. Tyla. ....                                  | 8  |
| Tabulka 1 | Rozpoznávání prostředí podle souboru popisujícího balíček. .... | 8  |
| Tabulka 2 | Počet vlastních skriptů před sjednocením a po něm. ....         | 10 |

# 1 Úvod

Vývoj softwaru se za posledních dvacet let z velké části zautomatizoval. Sestavení programu, spuštění testů i nasazení do provozu dnes obstarávají stroje. Jeden krok však zůstal ruční: samotná změna zdrojového kódu. Právě tam se tvoří většina prodlev — mezi tím, kdy někdo popíše požadavek, a tím, kdy se změna dostane k uživatelům.

Výrobní průmysl zná pojem *dark factory*, tedy „temná továrna“: provoz, který běží bez lidské obsluhy, a proto v něm nemusí svítit. Tato práce zkoumá, nakolik lze stejný princip uplatnit ve vývoji softwaru — a kde jsou jeho hranice.

## 1.1 Cíl práce

Cílem práce je navrhnout, realizovat a ověřit systém, který automatizuje vývojový proces od přijetí požadavku po vytvoření ověřené změny, aniž by se vzdal lidského schválení v rozhodujících bodech.

Dílčí cíle:

1. Popsat současný stav automatizace vývoje softwaru a orchestrace jazykových modelů.
2. Navrhnout architekturu systému, který provede požadavek celým procesem.
3. Systém realizovat a nasadit na reálné repozitáře.
4. Vyhodnotit jeho chování a pojmenovat omezení, na která v provozu narazil.

## 1.2 Struktura práce

Kapitola 2 shrnuje teoretická východiska. Kapitola 3 popisuje vlastní systém DarkFactory. Kapitola 4 hodnotí výsledky jeho nasazení a kapitola 5 je shrnuje.

## 2 Teoretická část

Tato kapitola vymezuje pojmy, o které se opírá praktická část: řízení verzí, kontinuální integrace a orchestrace jazykových modelů.

### 2.1 Řízení verzí a model pull requestu

Distribuované řízení verzí umožňuje vést změny odděleně od hlavní vývojové větve a slučovat je až po přezkoumání. Tento postup — *pull request* — je dnes ve vývoji softwaru převládající a je zároveň místem, kde se uplatňuje kontrola kvality.

### 2.2 Kontinuální integrace

Kontinuální integrace znamená, že se každá změna automaticky sestaví a otestuje. Zavádí pojem *požadované kontroly* (angl. *required checks*): množinu úloh, které musí skončit úspěšně, jinak změnu nelze sloučit. Tím se z kvality stává vlastnost vynucovaná strojem, nikoli dohodou.

### 2.3 Jazykové modely a jejich orchestrace

Velké jazykové modely dokážou generovat i upravovat zdrojový kód. Pro praktické nasazení je ovšem podstatnější než samotná schopnost generovat text to, jak je model zasazen do procesu: jaké dostane vstupy, jaké má k dispozici nástroje a kdo ověřuje jeho výstup.

#### 2.3.1 Problém jednoho poskytovatele

Spoléhá-li systém na jediného poskytovatele modelu, zastaví se ve chvíli, kdy tento poskytovatel vyčerpá kvótu nebo změni rozhraní. Řešením je vrstva, která umí tentýž úkol předat kterémukoli z několika poskytovatelů.

### 2.4 Governance automatizovaných změn

Čím je systém samostatnější, tím důležitější je otázka, kde do procesu vstupuje člověk. Úplná autonomie není cílem — cílem je autonomie v rutinních krocích a lidské rozhodnutí tam, kde je nevratné.

### 3 Praktická část

Praktickou částí práce je systém **DarkFactory** — sada pravidel, skriptů a pracovních postupů, které z repozitáře udělají samostatně pracující vývojový provoz. Zdrojový kód je veřejně dostupný (1).

#### 3.1 Přehled architektury

Systém je navržen tak, aby byl sdílen mezi repozitáři beze změny: pracovní postupy i skripty jsou ve všech repozitářích totožné a jediné, co se liší, je konfigurační soubor popisující daný repozitář.



Obrázek 1: Logo Gymnázia J. K. Tyla.

#### 3.2 Životní cyklus požadavku

Požadavek prochází systémem v pevně daných krocích, mezi nimiž jsou schvalovací body:

1. Uživatel založí požadavek s doslovným zněním svého zadání.
  2. Systém požadavek interpretuje a čeká na schválení této interpretace.
  3. Po schválení vzniká plán jako samostatný podřízený úkol.
  4. Po schválení plánu vzniká větev a návrh změny.
  5. Změna projde vlastním přezkoumáním a ověřením proti plánu.
- Doslovné znění zadání je uchováno záměrně: parafráze ztrácí význam, který do ní zadavatel vložil.

#### 3.3 Popis prostředí repozitáře

Aby systém věděl, které úlohy má spustit, musí rozpoznat, z čeho se repozitář skládá. Rozpoznávání vychází z názvů souborů popisujících balíček — například `pyproject.toml` značí Python, `Cargo.toml` jazyk Rust.

| Soubor                      | Prostředí | Doména |
|-----------------------------|-----------|--------|
| <code>pyproject.toml</code> | Python    | kód    |
| <code>Cargo.toml</code>     | Rust      | kód    |
| <code>package.json</code>   | Node      | kód    |
| <code>typst.toml</code>     | Typst     | text   |
| <code>.latexmkrc</code>     | LaTeX     | text   |

Tabulka 1: Rozpoznávání prostředí podle souboru popisujícího balíček.



Rozdělení na **domény** a **prostředí** dovoluje popsat i repozitář, který obsahuje zároveň program a text — jako právě tato práce, jejíž praktickou částí je software popisovaný v Tabulka 1.

### **3.4 Orchestrace napříč poskytovateli**

Systém neváže na jednoho poskytovatele modelu. Vyčerpá-li jeden z nich kvótu, předá se tentýž úkol dalšímu v pořadí, místo aby se proces zastavil.

### **3.5 Ověřování změn**

Každá změna musí projít požadovanými kontrolami. Ty jsou navrženy tak, aby vždy skončily nějakým výsledkem — úloha, která se může „přeskočit“, by jinak zablokovala slučování napořád.

## 4 Výsledky a diskuse

### 4.1 Nasazení

Systém byl nasazen na tři repozitáře. Sdílení jednoho pracovního postupu místo kopií vedlo k odstranění přibližně 7 800 řádků duplicitního kódu.

| Repozitář       | Skripty před | Skripty po |
|-----------------|--------------|------------|
| DarkFactory     | 12           | 12         |
| omnis           | 7            | 1          |
| ChessWithQuests | 5            | 1          |

Tabulka 2: Počet vlastních skriptů před sjednocením a po něm.

### 4.2 Chyby, které se projeví až v provozu

Nasazení odhalilo několik chyb, které se při návrhu neprojevily:

**Uvážnutí souběžnosti** Volající i volaný pracovní postup použily stejný název skupiny souběžnosti, takže volaný čekal na skupinu drženou vlastním volajícím. Běh skončil bez jediné úlohy a bez chybového hlášení.

**Přejmenování kontrol** Volaný pracovní postup hlásí výsledek pod složeným názvem. Ochrana větve vyžadující původní název by zablokovala každé sloučení.

**Tiché selhání zápisu na nástěnku** Zápisy na projektovou nástěnku byly obaleny zachycením všech výjimek, takže selhání skončilo hlášením o úspěchu. Chyba vyšla najevo až ručním porovnáním obsahu nástěnky.

### 4.3 Diskuse

Poslední z uvedených chyb je pro celou práci nejpoučnější. Systém, který své vlastní selhání zamlčí, je nebezpečnější než systém, který spadne: nespolehlivost je v něm neviditelná. Z toho plyne obecnější závěr — u automatizovaného provozu je hlášení chyb stejně důležitou vlastností jako samotná funkce.

### 4.4 Omezení

Systém předpokládá, že úkol lze ověřit strojově. Tam, kde správnost posoudí až člověk, zůstává přínos automatizace omezený.

## 5 Závěr

Cílem práce bylo navrhnout, realizovat a ověřit systém automatizující vývojový proces od přijetí požadavku po vytvoření ověřené změny. Cíl byl naplněn: systém DarkFactory byl realizován a nasazen na tři repozitáře, kde odstranil přibližně 7 800 řádků duplicitního kódu a převzal rutinní kroky vývojového procesu.

Ukázalo se, že hlavní obtíž nespočívá v generování změn, ale v jejich řízení — v tom, kde do procesu vstupuje člověk a jak systém hlásí vlastní selhání. Chyby nalezené při nasazení se týkaly výhradně této oblasti, nikoli schopnosti změnu vytvořit.

Další rozvoj se nabízí ve dvou směrech: rozšířit popis prostředí i na jiné než programové domény, tedy na sazbu textů, a doplnit hlášení chyb tak, aby každé selhání zanechalo trvalý a viditelný záznam.

## **Seznam příloh**

|                                          |    |
|------------------------------------------|----|
| A Obsah přiloženého média .....          | 13 |
| B Struktura konfiguračního souboru ..... | 14 |

## **A Obsah přiloženého média**

Odevzdaný archiv obsahuje zdrojové soubory této práce a odkaz na veřejný repozitář se zdrojovým kódem systému DarkFactory.

## **B Struktura konfiguračního souboru**

Ukázka konfigurace popisující jeden repozitář sdílenému pracovnímu postupu.

## Seznam zdrojů

1. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>